Rockchip Baseparameter 分区格式定义和使用说明

文件标识：RK-YH-YF-194

发布版本：V1.2.0

日期：2026-01-06

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

产品版本

芯片名称	内核版本	**Baseparameter** 版本
RK3288	Linux kernel 4.4 及以上内核	V1.0
RK3399	Linux kernel 4.4 及以上内核	V1.0
RK356X	Linux kernel 4.19 及以上内核	V2.0
RK3588	Linux kernel 5.10 及以上内核	V2.0
RK3528	Linux kernel 4.19 及以上内核	V2.0
RK3562	Linux kernel 5.10 及以上内核	V2.0
RK3576	Linux kernel 6.1 及以上内核	V2.1
RK3538	Linux kernel 6.1 及以上内核	V3.0

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	黄家钗/林垚	2021-08-27	初始版本
V1.1.0	丁凌崧/林垚	2025-11-05	更新 baseparameter V2.1 版本相关内容
V1.2.0	丁凌崧/林垚/吴方熠	2026-01-06	更新 baseparameter V1.0 和 V3.0 版本相关内容

目录

Rockchip Baseparameter 分区格式定义和使用说明

1. Baseparameter 概述
   1.1 版本更新说明
2. 分区拓扑图
   2.1 Baseparameter V1
   2.2 Baseparameter V2
   2.3 Baseparameter V3
3. Baseparameter 结构
   3.1 Baseparameter V1
      3.1.1 struct bp_disp_info_v1
         3.1.1.1 struct bp_screen_info_v1
         3.1.1.2 struct bp_overscan_info
         3.1.1.3 struct bp_hwc_initial_info
         3.1.1.4 struct bp_bcsh_info
         3.1.1.5 u32 reserved[128]
         3.1.1.6 struct bp_lut_data
   3.2 Baseparameter V2
      3.2.1 struct disp_header
      3.2.2 struct bp_disp_info_v2
         3.2.2.1 struct bp_screen_info_v2
         3.2.2.2 struct bp_bcsh_info
         3.2.2.3 struct bp_overscan_info
         3.2.2.4 struct bp_gamma_lut_data
         3.2.2.5 struct bp_cubic_lut_data
         3.2.2.6 struct bp_framebuffer_info
         3.2.2.7 u32 reserved[244]
      3.2.3 struct bp_pq_tuning_info_v2
         3.2.3.1 struct bp_csc_info_v2
         3.2.3.2 struct bp_dci_info
         3.2.3.3 struct bp_acm_info
         3.2.3.4 struct bp_gamma_lut_data
      3.2.4 struct bp_pq_factory_info
         3.2.4.1 struct bp_bcsh_info
         3.2.4.2 struct bp_white_balance_info
      3.2.5 struct bp_pq_sharp_info
      3.2.6 struct bp_aipq_info
   3.3 Baseparamter V3
      3.3.1 struct bp_disp_info_v3
         3.3.1.1 struct bp_screen_info_v2
         3.3.1.2 struct bp_pq_tuning_info_v3
            3.3.1.2.1 struct bp_csc_info_v3
   3.4 占用存储空间
      3.4.1 Baseparameter V1
      3.4.2 Baseparameter V2
      3.4.3 Baseparameter V3
4. Baseparameter 备份分区
5. Baseparameter 读写 API 说明
6. Baseparameter 解析工具使用说明
7. Android 软件配置流程和代码路径

1. Baseparameter 概述

Rockchip 平台 Baseparameter 分区用于存储各显示通路（显示通路概念详见文档《Rockchip_Developer_Guide_DRM_Display_Driver_CN》的《Display feature》章节）定制化的分辨率、BCSH 参数、Gamma Lut 配置、3D Lut 配置、PQ Tuning 参数（包含 ACM/CSC/DCI/SHP 等 feature 的配置参数）等信息：

Feature	**Introduction**
3D Lut（3D Lookup Table）	RGB 色彩映射
ACM（Auto Color Management）	色彩管理，改变目标颜色的亮度、色调、饱和度
BCSH（Brightness/Contrast/Saturation/Hue）	调整画面整体的亮度、对比度、饱和度和色调
CSC（Corlor Space Convert）	YUV 和 RGB 色域转换以及 Full-Range 和 Limited-Range 变换
DCI（Dynamic Contrast Improvement）	全局动态对比度调整
Gamma Lut（Gamma Lookup Table）	显示输出 Gamma 映射
SHP（Sharpen）	全局清晰度调整

当 Baseparameter 分区固件不包含有效的上述显示配置信息时（**默认 SDK Baseparameter 固件通常为 dummy**），基础分辨率及各显示效果参数由应用层通过 libdrm 提供的接口或 Rockchip PQTool 进行调节（参考文档《Rockchip_PQTool_Guide》）。

Baseparameter 分区的主要功能如下：

- 应用层通过 libbaseparameter 提供的接口（详见下文《Baseparameter 读写 API 说明》），可实现 Baseparameter 分区显示配置信息的读写，**进而将用户定制化的显示效果配置参数存于存储介质中**。
- 开启 Logo Display 功能（详见文档《Rockchip_Developer_Guide_DRM_Display_Driver_CN》的《U-Boot 驱动》章节）时，U-Boot Baseparameter 驱动能够解析并加载 Baseparameter 分区中存储的各显示配置，**实现定制化的 Logo 显示效果调节功能**。

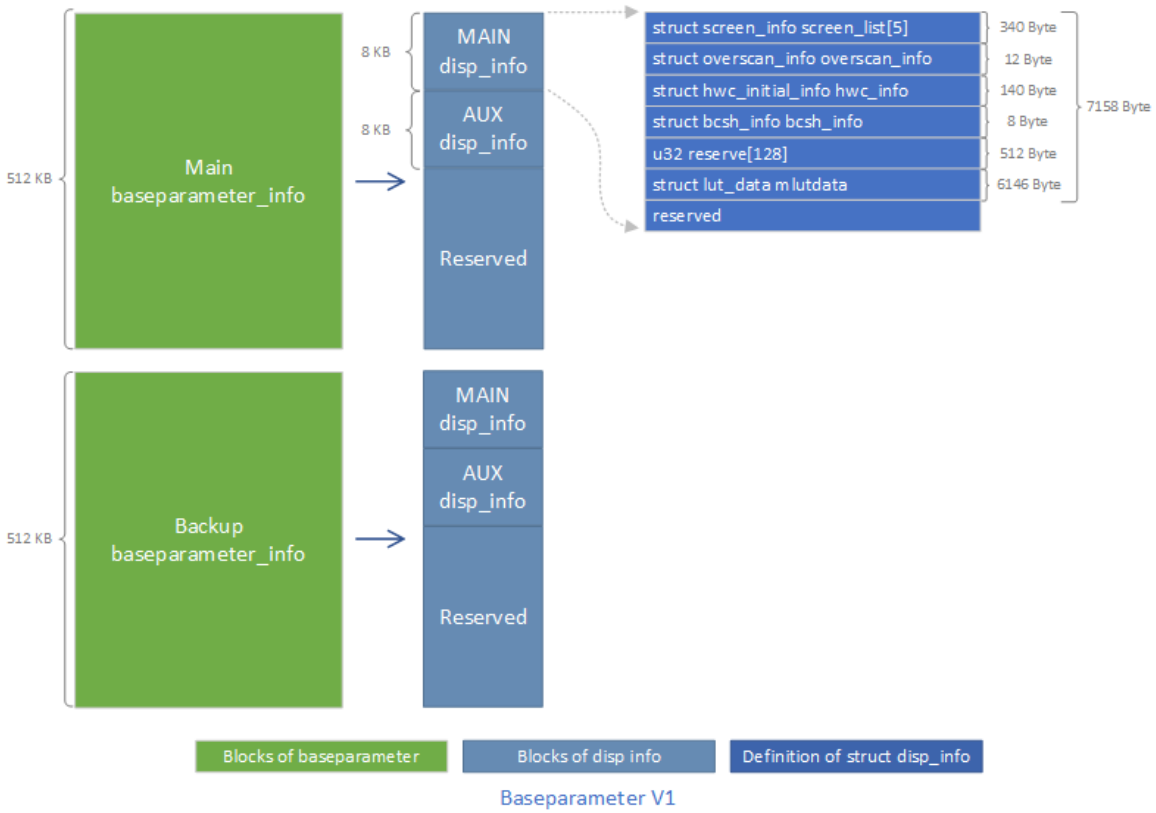
1.1 版本更新说明

Baseparameter Version	更新内容
V1.0	适配 VOP V1.0 的基础版本
V2.0	适配 VOP V2.0 的基础版本
V2.1	添加 PQ 相关的结构体 struct pq_tuning_info/struct pq_factory_info/struct pq_sharp_info/struct aipq_info
V3.0	1. 将 V2.1中新增的 PQ 相关结构体移动到 struct disp_info 中，以拓展多路 PQ 功能。 - struct pq_tuning_info/struct pq_factory_info 合入 struct disp_info。 - struct pq_sharp_info 和 struct aipq_info 合入 struct pq_tuning_info。 3. struct disp_info 中删除多余的 reserved 数组。

Baseparameter Version	4. struct disp_info 中添加 u32 reserved_offset 以指明 reserved 空间的地址偏移。 5. struct csc_info 结构新增参数 csc_r_offset/csc_g_offset/csc_b_offset。 6. struct screen_info 中 feature 参数新增 ALLM_EN 选项。
	更新内容

2. 分区拓扑图

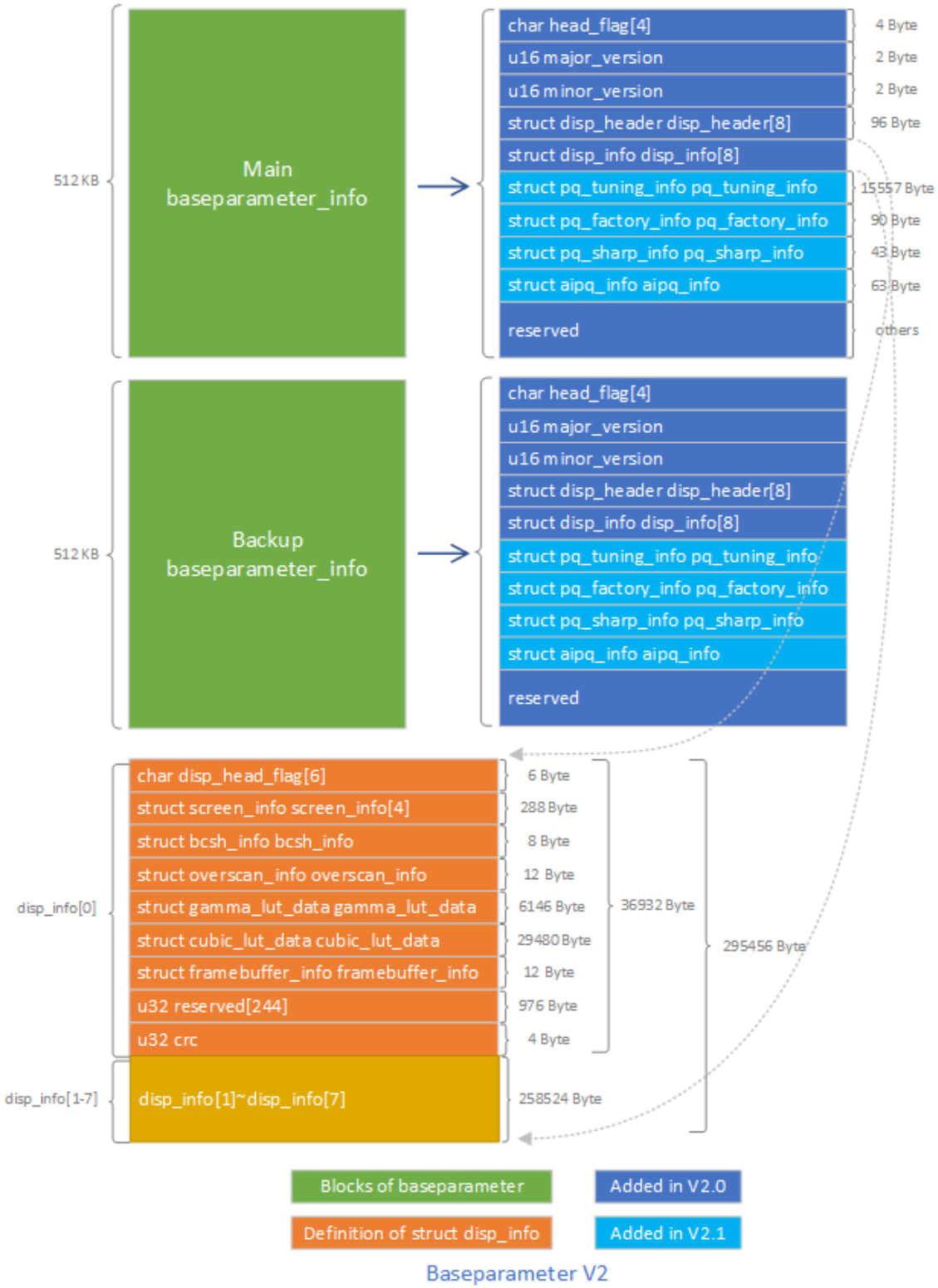
2.1 Baseparameter V1



Baseparameter V1

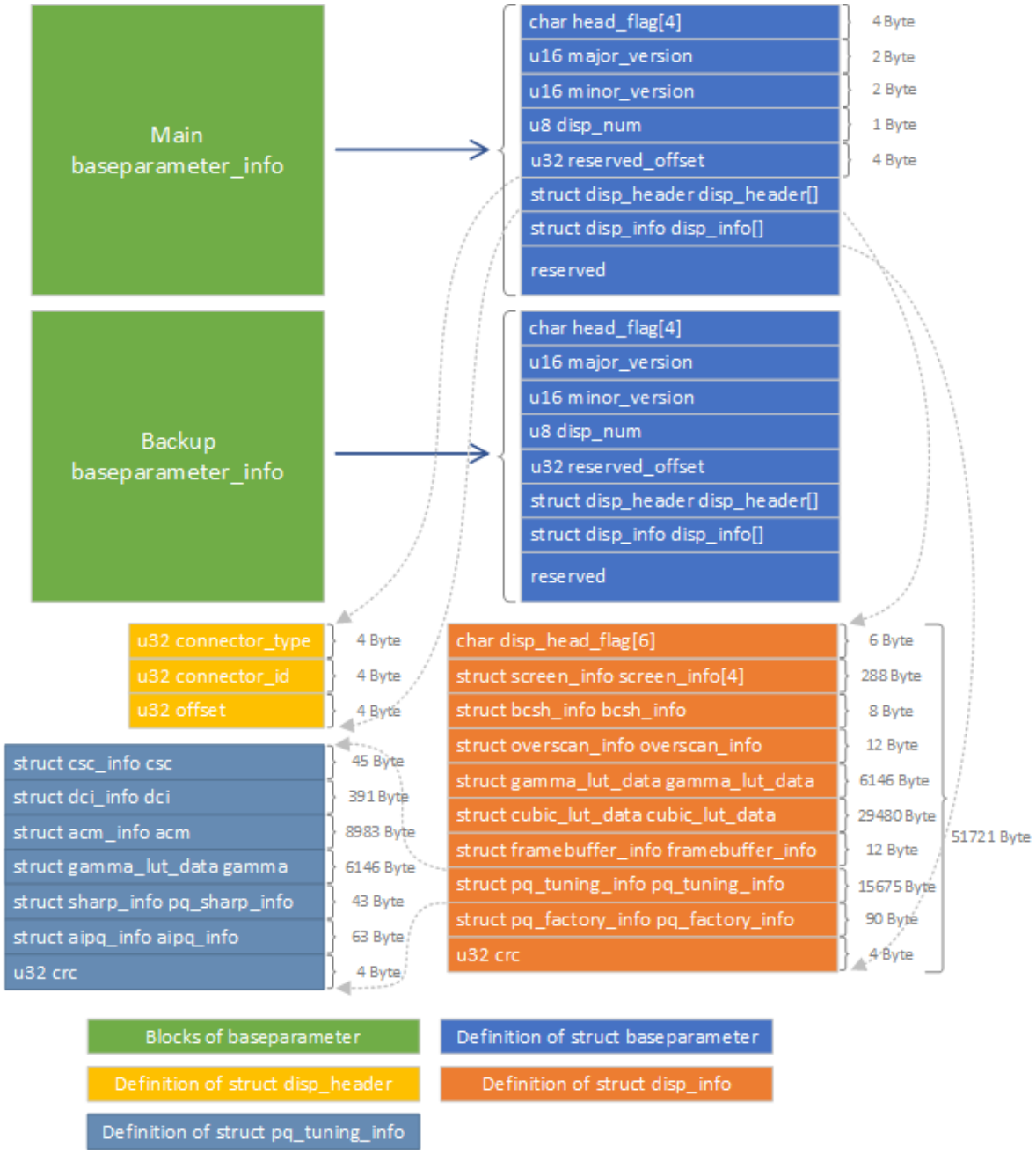
- Main baseparameter_info：存储用户通过应用层保存的显示配置信息。
  - MAIN disp_info：主屏的分辨率、过扫描配置、BCSH 参数和 Gamma Lut 配置等。
  - AUX disp_info：副屏的上述配置。
- Backup baseparameter_info：出厂固件默认的显示配置信息，在恢复出厂设置时将覆盖 Main baseparameter_info。

2.2 Baseparameter V2



- Main baseparameter_info：存储用户通过应用层保存的显示配置信息，包含各显示通路的分辨率、BCSH 参数、Gamma Lut 配置、3D Lut 配置、PQ Tuning 参数等。
- Backup baseparameter_info：出厂固件默认的显示配置信息，在恢复出厂设置时将覆盖 Main baseparameter_info。

2.3 Baseparameter V3



Baseparameter V3

- Main baseparameter_info：存储用户通过应用层保存的显示配置信息，包含各显示通路的分辨率、BCSH 参数、Gamma Lut 配置、3D Lut 配置、PQ Tuning 参数等。
- Backup baseparameter_info：出厂固件默认的显示配置信息，在恢复出厂设置时将覆盖 Main baseparameter_info。

3. Baseparameter 结构

U-Boot 驱动 drivers/video/drm/rockchip_baseparameter.c 支持上述的 Baseparamter V1/V2/V3 版本，本章节后文中各结构体及代码介绍均基于 U-Boot Baseparamter 驱动展开说明。

3.1 Baseparameter V1

```
struct baseparameter_info_v1 {
    struct bp_disp_info_v1 main;     /* 主屏显示配置信息，size：7158 Byte */
    struct bp_disp_info_v1 aux;      /* 副屏显示配置信息，size：7158 Byte */
};
```

3.1.1 struct bp_disp_info_v1

```
struct bp_disp_info_v1 {
    /* connector-mirror 模式下各接口的显示配置, size: 68 * 5 = 340 Byte */
    struct bp_screen_info_v1 screen_info[BP_V1_SCREEN_INFO_ARRAY_SIZE];
    struct bp_overscan_info overscan_info;  /* 过扫描信息，size：12 bytes */
    struct bp_hwc_initial_info hwc_info;    /* HWC 初始化信息，size：140 bytes */
    struct bp_bcsh_info bcsh_info;          /* 调节亮度、对比度、饱和度、色度信息，
size：8 Byte */
    char reserve[512];                      /* 预留，size：512 Byte */
    struct bp_lut_data mlutdata;            /* lut 信息，size：6146 Byte */
};
```

主屏显示配置信息，占用存储空间：7158 Byte。

3.1.1.1 struct bp_screen_info_v1

```
struct bp_screen_info_v1 {
    u32 type;
    struct bp_display_mode mode;    /* 52 bytes */
    enum bp_output_format format;   /* 4 bytes */
    enum bp_output_depth depth; /* 4 bytes */
    u32 feature;
};
```

显示接口的类型、分辨率时序等信息，占用 68 Byte。

- type：屏的接口类型。

```
#define DRM_MODE_CONNECTOR_Unknown  0
#define DRM_MODE_CONNECTOR_VGA      1
#define DRM_MODE_CONNECTOR_DVII     2
#define DRM_MODE_CONNECTOR_DVID     3
#define DRM_MODE_CONNECTOR_DVIA     4
#define DRM_MODE_CONNECTOR_Composite    5
#define DRM_MODE_CONNECTOR_SVIDEO   6
#define DRM_MODE_CONNECTOR_LVDS     7
#define DRM_MODE_CONNECTOR_Component    8
#define DRM_MODE_CONNECTOR_9PinDIN  9
#define DRM_MODE_CONNECTOR_DisplayPort  10
#define DRM_MODE_CONNECTOR_HDMIA    11
#define DRM_MODE_CONNECTOR_HDMIB    12
#define DRM_MODE_CONNECTOR_TV       13
#define DRM_MODE_CONNECTOR_eDP      14
#define DRM_MODE_CONNECTOR_VIRTUAL  15
#define DRM_MODE_CONNECTOR_DSI      16
#define DRM_MODE_CONNECTOR_DPI      17
```

```
#define DRM_MODE_CONNECTOR_WRITEBACK    18
```

- resolution：扫描时序信息。

```
struct bp_display_mode {
    int clock;      /* in kHz */
    int hdisplay;
    int hsync_start;
    int hsync_end;
    int htotal;
    int vdisplay;
    int vsync_start;
    int vsync_end;
    int vtotal;
    int vrefresh;
    int vscan;
    u32 flags;
    int picture_aspect_ratio;
};
```

- output_format: 颜色格式，属性字符串为 Auto 时，选择output_ycbcr_high_subsampling。

```
enum bp_output_format {
    RK_IF_FORMAT_RGB,   /* default RGB */
    RK_IF_FORMAT_YCBCR444,  /* YCBCR 444 */
    RK_IF_FORMAT_YCBCR422,  /* YCBCR 422 */
    RK_IF_FORMAT_YCBCR420,  /* YCBCR 420 */
    /* (YCbCr444 > YCbCr422 > YCbCr420 > RGB) */
    RK_IF_FORMAT_YCBCR_HQ,  /* Highest subsampled YUV */
    /* (YCbCr420 > YCbCr422 > YCbCr444 > RGB) */
    RK_IF_FORMAT_YCBCR_LQ,  /* Lowest subsampled YUV */
    RK_IF_FORMAT_MAX,
};
```

- depth：色深，属性字符串为 Auto 时，选择 Automatic。

```
enum bp_output_depth {
    AUTOMATIC = 0,
    DEPTH_24BIT = 8,
    DEPTH_30BIT = 10,
};
```

- feature：配置分辨率以及颜色的 Auto 模式、是否开启 DPCP1X 以及是否过滤分辨率列表。

```
#define RESOLUTION_AUTO         (1<<0)
#define COLOR_AUTO              (1<<1)
#define HDCP1X_EN               (1<<2)
#define RESOLUTION_WHITE_EN     (1<<3)
```

3.1.1.2 struct bp_overscan_info

过扫描缩放系数。

```
struct bp_overscan_info {
    u32 maxvalue;
    u16 leftscale;
    u16 rightscale;
    u16 topscale;
    u16 bottomscale;
};
```

3.1.1.3 struct bp_hwc_initial_info

HWC 初始化配置信息，包含 framebuffer 的 size、帧率和挂载的设备名称等。

```
struct bp_hwc_initial_info {
    char device[128];
    u32 framebuffer_width;
    u32 framebuffer_height;
    float fps;
};
```

3.1.1.4 struct bp_bcsh_info

BCSH 信息，取值范围 0~100，默认值均为 50，用于调节亮度、对比度、饱和度、色度。

```
struct bp_bcsh_info {
    u16 brightness;
    u16 contrast;
    u16 saturation;
    u16 hue;
};
```

3.1.1.5 u32 reserved[128]

预留临时配置数据的空间。

3.1.1.6 struct bp_lut_data

保存 LUT 表数据，size 表示每个分量 lut 表中有多少个数据，最大为 1024。

```
struct bp_lut_data {
    u16 size;
    u16 lred[1024];
    u16 lgreen[1024];
    u16 lblue[1024];
};
```

3.2 Baseparameter V2

```
struct baseparameter_info_v2 {
    char head_flag[4]; /* 头标识，"BASP" */
    u16 major_version; /* Baseparameter 大版本信息 */
    u16 minor_version; /* Baseparameter 小版本信息 */
    /* 通过 header 可以正确找到每一个显示设备的偏移，按现在每个 disp_info 的大小，最多支持
8 个 disp */
    struct bp_disp_header disp_header[BP_V2_DISP_INFO_ARRAY_SIZE];
    struct bp_disp_info_v2 disp_info[BP_V2_DISP_INFO_ARRAY_SIZE]; /* 显示设备信息
*/

    /* V2.1 版本新增配置信息 */
    struct bp_pq_tuning_info_v2 pq_tuning_info; /* PQ tuning 数据 */
    struct bp_pq_factory_info pq_factory_info;  /* 工厂存储的 PQ 数据 */
    struct bp_pq_sharp_info pq_sharp_info;      /* PQ sharp 数据 */
    struct bp_aipq_info aipq_info;              /* AI PQ 数据 */
};
```

3.2.1 struct disp_header

```
struct bp_disp_header {
    u32 connector_type; /* 显示设备类型 */
    u32 connector_id;   /* 显示设备 id */
    /* the memory offset based &baseparameter_info of disp info for specific
connector */
    u32 offset;         /* disp_info 偏移 */
};
```

占用存储空间： 12 Byte。

3.2.2 struct bp_disp_info_v2

```
struct bp_disp_info_v2 {
    char disp_head_flag[6]; /* disp 头标识，"DISP_N"，N 可以是 0-7，size：6 Byte */
    /* connector-mirror 模式下各接口的显示配置，size：72 * 4 = 288 Byte */
    struct bp_screen_info_v2 screen_info[BP_V2_SCREEN_INFO_ARRAY_SIZE];
    struct bp_bcsh_info bcsh_info; /* 调节亮度、对比度、饱和度、色度信息，size：8 Byte
*/
    struct bp_overscan_info overscan_info; /* 过扫描信息，size：12 Byte */
    struct bp_gamma_lut_data gamma_lut_data; /* gamma 信息，size：6146 Byte */
    struct bp_cubic_lut_data cubic_lut_data; /* 3D lut 信息，size：29480 Byte */
    struct bp_framebuffer_info framebuffer_info; /* framebuffer 信息，size：12
Byte */
    u32 reserved[244]; /* 预留，size：976 Byte */
    u32 crc; /* CRC 校验，size：4 Byte */
};
```

占用存储空间：36932 Byte。

3.2.2.1 struct bp_screen_info_v2

```
struct bp_screen_info_v2 {
    u32 type;   /* connector 类型，4 bytes */
    u32 id; /* 4 byte，用于区别相同 type 的不同设备 */
    struct bp_display_mode mode;    /* 52 bytes */
    enum bp_output_format format;   /* 4 bytes */
    enum bp_output_depth depth; /* 4 bytes */
    u32 feature;    /* 4 bytes */
};
```

显示接口的类型、分辨率时序等信息，占用 72 Byte。

- type：屏的接口类型。

```
#define DRM_MODE_CONNECTOR_Unknown  0
#define DRM_MODE_CONNECTOR_VGA      1
#define DRM_MODE_CONNECTOR_DVII     2
#define DRM_MODE_CONNECTOR_DVID     3
#define DRM_MODE_CONNECTOR_DVIA     4
#define DRM_MODE_CONNECTOR_Composite    5
#define DRM_MODE_CONNECTOR_SVIDEO   6
#define DRM_MODE_CONNECTOR_LVDS     7
#define DRM_MODE_CONNECTOR_Component    8
#define DRM_MODE_CONNECTOR_9PinDIN  9
#define DRM_MODE_CONNECTOR_DisplayPort  10
#define DRM_MODE_CONNECTOR_HDMIA    11
#define DRM_MODE_CONNECTOR_HDMIB    12
#define DRM_MODE_CONNECTOR_TV       13
#define DRM_MODE_CONNECTOR_eDP      14
#define DRM_MODE_CONNECTOR_VIRTUAL  15
#define DRM_MODE_CONNECTOR_DSI      16
#define DRM_MODE_CONNECTOR_DPI      17
#define DRM_MODE_CONNECTOR_WRITEBACK    18
```

- resolution：扫描时序信息。

```
struct bp_display_mode {
    int clock;      /* in kHz */
    int hdisplay;
    int hsync_start;
    int hsync_end;
    int htotal;
    int vdisplay;
    int vsync_start;
    int vsync_end;
    int vtotal;
    int vrefresh;
    int vscan;
    u32 flags;
    int picture_aspect_ratio;
};
```

- output_format: 颜色格式，属性字符串为 Auto 时，选择output_ycbcr_high_subsampling。

```
enum bp_output_format {
    RK_IF_FORMAT_RGB,    /* default RGB */
    RK_IF_FORMAT_YCBCR444,  /* YCBCR 444 */
    RK_IF_FORMAT_YCBCR422,  /* YCBCR 422 */
    RK_IF_FORMAT_YCBCR420,  /* YCBCR 420 */
    /* (YCbCr444 > YCbCr422 > YCbCr420 > RGB) */
    RK_IF_FORMAT_YCBCR_HQ,  /* Highest subsampled YUV */
    /* (YCbCr420 > YCbCr422 > YCbCr444 > RGB) */
    RK_IF_FORMAT_YCBCR_LQ,  /* Lowest subsampled YUV */
    RK_IF_FORMAT_MAX,
};
```

- depth：色深，属性字符串为 Auto 时，选择 Automatic。

```
enum bp_output_depth {
    AUTOMATIC = 0,
    DEPTH_24BIT = 8,
    DEPTH_30BIT = 10,
};
```

- feature：配置分辨率以及颜色的 Auto 模式、是否开启 DPCP1X 以及是否过滤分辨率列表。

```
#define RESOLUTION_AUTO          (1<<0)
#define COLOR_AUTO               (1<<1)
#define HDCP1X_EN                (1<<2)
#define RESOLUTION_WHITE_EN      (1<<3)
```

- id：同类型显示接口的 index。以 RK3588 平台为例，该参数用以区分 HDMI0/HDMI1、DP0/DP1 和 eDP0/eDP1 等。

3.2.2.2 struct bp_bcsh_info

同上述《struct bp_bcsh_info》章节。

3.2.2.3 struct bp_overscan_info

同上述《struct bp_overscan_info》章节。

3.2.2.4 struct bp_gamma_lut_data

保存 gamma lut data 信息，size 表示每个分量 lut 表中有多少个数据，最大为 1024。

```
#define BP_GAMMA_LUT_DATA_ARRAY_SIZE    (1024)

struct bp_gamma_lut_data {
    u16 size;
    u16 lred[BP_GAMMA_LUT_DATA_ARRAY_SIZE];
    u16 lgreen[BP_GAMMA_LUT_DATA_ARRAY_SIZE];
    u16 lblue[BP_GAMMA_LUT_DATA_ARRAY_SIZE];
};
```

3.2.2.5 struct bp_cubic_lut_data

保存 cubic lut data 信息，size 表示每个分量 lut 表中有多少个数据，最大为 17x17x17 = 4913。

```
#define BP_CUBIC_LUT_DATA_ARRAY_SIZE    (4913)

struct bp_cubic_lut_data {
    u16 size;
    u16 lred[BP_CUBIC_LUT_DATA_ARRAY_SIZE];
    u16 lgreen[BP_CUBIC_LUT_DATA_ARRAY_SIZE];
    u16 lblue[BP_CUBIC_LUT_DATA_ARRAY_SIZE];
};
```

3.2.2.6 struct bp_framebuffer_info

预设 framebuffer 信息。

```
struct bp_framebuffer_info {
    u32 framebuffer_width;
    u32 framebuffer_height;
    u32 fps;
};
```

3.2.2.7 u32 reserved[244]

预留临时配置数据的空间。

3.2.3 struct bp_pq_tuning_info_v2

PQ 显示效果配置信息，Baseparameter V2.1 新增。

```
struct bp_pq_tuning_info_v2 {
    struct bp_csc_info_v2 csc_info; /* CSC 配置，size：33 Byte */
    struct bp_dci_info dci_info;    /* DCI 配置，size：391 Byte  */
    struct bp_acm_info acm_info;    /* ACM 配置，size：8983 Byte  */
    struct bp_gamma_lut_data gamma_lut_data;    /* gamma 信息，size：6146 Byte  */
    u32 crc;    /* CRC 校验，size：4 Byte */
};
```

占用存储空间：15557 Byte。

3.2.3.1 struct bp_csc_info_v2

保存 CSC 色彩空间转换配置信息。

```
struct bp_csc_info_v2 {
    bool csc_enable;
    enum bp_csc_mode mode;
    u32 csc_brightness;
    u32 csc_contrast;
    u32 csc_saturation;
    u32 csc_hue;
    u32 csc_r_gain;
    u32 csc_g_gain;
    u32 csc_b_gain;
};
```

- mode：CSC 模式选择：

```
enum bp_csc_mode {
    HIGH_QUALITY_MODE = 0,
    LOW_LATENCY_MODE = 1,
};
```

3.2.3.2 struct bp_dci_info

保存 DCI 显示效果配置信息。

```
struct bp_dci_info {
    bool dci_enable;
    u16 dci_wgt_coef_low[33];
    u16 dci_wgt_coef_mid[33];
    u16 dci_wgt_coef_high[33];
    u16 dci_weight_low[32];
    u16 dci_weight_mid[32];
    u16 dci_weight_high[32];
};
```

3.2.3.3 struct bp_acm_info

保存 ACM 自适应色彩管理配置信息。

```
struct bp_acm_info {
    bool acm_enable;
    s16 acm_table_delta_yby_h[65];
    s16 acm_table_delta_hby_h[65];
    s16 acm_table_delta_sby_h[65];
    s16 acm_table_gain_yby_y[585];
    s16 acm_table_gain_hby_y[585];
    s16 acm_table_gain_sby_y[585];
    s16 acm_table_gain_yby_s[845];
    s16 acm_table_gain_hby_s[845];
    s16 acm_table_gain_sby_s[845];
    u32 lum_gain;
    u32 hue_gain;
    u32 sat_gain;
};
```

3.2.3.4 struct bp_gamma_lut_data

同上述《struct bp_gamma_lut_data》章节。

3.2.4 struct bp_pq_factory_info

产线测试使用的 PQ 显示效果配置信息，Baseparameter V2.1 新增。

```
struct bp_pq_factory_info {
    struct bcsh_info bcsh[4]; /* 调节亮度、对比度、饱和度、色度信息，size: 8 * 4 = 32
Byte */
    struct bp_white_balance_info white_balance[4]; /* 白平衡信息，size: 12 * 4 = 48
Byte */
    u8 cur_bcsh_index; /* 当前使用的 bcsh 数据索引，size: 1 Byte */
    u8 cur_white_balance_index; /* 当前使用的白平衡数据索引，size: 1 Byte */
    u8 cur_dci_index; /* 当前使用的 DCI 数据索引，size: 1 Byte */
    u8 cur_acm_index; /* 当前使用的 ACM 数据索引，size: 1 Byte */
    u8 cur_gamma_index; /* 当前使用的 gamma 数据索引，size: 1 Byte */
    u8 cur_cubic_index; /* 当前使用的 3D lut 数据索引，size: 1 Byte */
    u32 crc; /* CRC 校验，size: 4 Byte */
};
```

占用存储空间：90 Byte。

3.2.4.1 struct bp_bcsh_info

同上述《struct bp_bcsh_info》章节。

3.2.4.2 struct bp_white_balance_info

保存白平衡配置信息。

```
struct bp_white_balance_info {
    u32 rgain;
    u32 ggain;
    u32 bgain;
};
```

3.2.5 struct bp_pq_sharp_info

PQ 锐化效果配置信息，Baseparameter V2.1 新增。

```
struct bp_pq_sharp_info {
    bool sharp_enable;
    u32 sharp_peaking_gain;
    bool sharp_enable_shoot_ctrl;
    u32 sharp_shoot_ctrl_over;
    u32 sharp_shoot_ctrl_under;
    bool sharp_enable_coring_ctrl;
```

```
    u16 sharp_coring_ctrl_ratio[4];
    u16 sharp_coring_ctrl_zero[4];
    u16 sharp_coring_ctrl_thrd[4];
    bool sharp_enable_gain_ctrl;
    u16 sharp_gain_ctrl_pos[4];
    bool sharp_enable_limit_ctrl;
    u16 sharp_limit_ctrl_pos0[4];
    u16 sharp_limit_ctrl_pos1[4];
    u16 sharp_limit_ctrl_bnd_pos[4];
    u16 sharp_limit_ctrl_ratio[4];
    u8 cur_sharp_index;
    u32 sharp_peaking_gain_mode[4];
    u8 cur_sharp_peaking_gain_mode_index;
    u32 crc;
};
```

占用存储空间：43 Byte。

3.2.6 struct bp_aipq_info

AI PQ 配置信息，Baseparameter V2.1 新增。

```
struct bp_aipq_info {
    bool ai_sd_enable;
    bool ai_sr_enable;
    u32 ai_sr_fix_model_idx;
    bool ai_sr_tuning_enable;
    u32 ai_sr_usm_gain_natural;
    bool ai_sr_usm_enable_ctrl_natural;
    u32 ai_sr_usm_ctrl_over_natural;
    u32 ai_sr_usm_ctrl_under_natural;
    u32 ai_sr_fusion_gain_natural;
    bool ai_sr_fusion_enable_ctrl_natural;
    u32 ai_sr_fusion_ctrl_over_natural;
    u32 ai_sr_fusion_ctrl_under_natural;
    u32 ai_sr_usm_gain_textual;
    bool ai_sr_usm_enable_ctrl_textual;
    u32 ai_sr_usm_ctrl_over_textual;
    u32 ai_sr_usm_ctrl_under_textual;
    u32 ai_sr_fusion_gain_textual;
    bool ai_sr_fusion_enable_ctrl_textual;
    u32 ai_sr_fusion_ctrl_over_textual;
    u32 ai_sr_fusion_ctrl_under_textual;
    u32 crc;
};
```

占用存储空间：63 Byte。

3.3 Baseparamter V3

```
struct baseparameter_info_v3 {
    char head_flag[4];  /* 头标识，"BASP" */
    u16 major_version;  /* Baseparameter 大版本信息 */
    u16 minor_version;  /* Baseparameter 小版本信息 */
    u8 disp_num;    /* 显示通路数量，对应 disp_info 数组大小 */
    u32 reserved_offset;    /* reserved 空间的地址偏移 */
    struct bp_disp_header *disp_header; /* 通过head可以正确找到每一个显示设备的偏移，按现在每个disp_info的大小，最多支持8个disp */
    struct bp_disp_info_v3 *disp_info;  /* 显示设备信息 */
};
```

（下文仅列出有相较 **V2** 有改动的结构）

3.3.1 struct bp_disp_info_v3

```
struct bp_disp_info_v3 {
    char disp_head_flag[6]; /* disp 头标识，"DISP_N"，N 可以是0-7，size: 6 Byte */
    struct bp_screen_info_v2 screen_info[BP_V2_SCREEN_INFO_ARRAY_SIZE]; /* connector-mirror 模式下各接口的显示配置，size: 72 * 4 = 288 Byte */
    struct bp_bcsh_info bcsh_info;  /* 调节亮度、对比度、饱和度、色度信息，size: 8 Byte */
    struct bp_overscan_info overscan_info;  /* 过扫描信息，size: 16 Byte */
    struct bp_gamma_lut_data gamma_lut_data;    /* gamma 信息，size: 6146 Byte */
    struct bp_cubic_lut_data cubic_lut_data;    /* 3D lut 信息，size: 29480 Byte */
    struct bp_framebuffer_info framebuffer_info;    /* framebuffer 信息，size: 12 Byte */
    struct bp_pq_tuning_info_v3 pq_tuning_info; /* PQ tuning 数据，size: 15675 Byte */
    struct bp_pq_factory_info pq_factory_info;  /* 工厂存储的 PQ 数据，size: 90 Byte */
    u32 crc;    /* CRC 校验，size: 4 Byte */
};
```

占用存储空间：51721 Byte。

3.3.1.1 struct bp_screen_info_v2

显示接口的类型、分辨率时序等信息，占用 72 Byte。

```
struct bp_screen_info_v2 {
    u32 type;   /* connector 类型，4 bytes */
    u32 id; /* 4 byte，用于区别相同 type 的不同设备 */
    struct bp_display_mode mode;    /* 52 bytes */
    enum bp_output_format format;   /* 4 bytes */
    enum bp_output_depth depth; /* 4 bytes */
    u32 feature;    /* 4 bytes */
};
```

- type/id/resolution/format/depth 同 V2，详见《struct bp_screen_info_v2》。
- feature：配置分辨率以及颜色的 Auto 模式、HDCP1X 模式开启 、分辨率白名单功能使能以及 ALLM 模式的开启。

```
#define RESOLUTION_AUTO          (1<<0)
#define COLOR_AUTO               (1<<1)
#define HDCP1X_EN                (1<<2)
#define RESOLUTION_WHITE_EN      (1<<3)
#define ALLM_EN                  (1<<4)
```

3.3.1.2 struct bp_pq_tuning_info_v3

```
struct bp_pq_tuning_info_v3 {
    struct bp_csc_info_v2 csc_info; /* CSC 配置，size: 45 Byte */
    struct bp_dci_info dci_info;    /* DCI 配置，size: 391 Byte  */
    struct bp_acm_info acm_info;    /* ACM 配置，size: 8983 Byte  */
    struct bp_gamma_lut_data gamma_lut_data;    /* gamma 信息，size: 6146 Byte  */
    struct bp_pq_sharp_info pq_sharp_info;  /* sharp 配置，size: 43 Byte */
    struct bp_aipq_info aipq_info;  /* AI PQ 配置，size: 63 Byte */
    u32 crc;    /* CRC 校验，size: 4 Byte */
};
```

struct dci_info/struct acm_info/struct gamma_lut_data/struct sharp_info/struct aipq_info 详见《struct bp_pq_tuning_info_v2》、《struct bp_pq_sharp_info》和《struct bp_aipq_info》，占用存储空间：15675 Byte。

3.3.1.2.1 struct bp_csc_info_v3

保存 CSC 色彩空间转换配置信息。

```
struct bp_csc_info_v3 {
    bool csc_enable;
    enum bp_csc_mode mode;
    u32 csc_brightness;
    u32 csc_contrast;
    u32 csc_saturation;
    u32 csc_hue;
    u32 csc_r_gain;
    u32 csc_g_gain;
    u32 csc_b_gain;
    u32 csc_r_offset;
    u32 csc_g_offset;
    u32 csc_b_offset;
};
```

3.4 占用存储空间

3.4.1 Baseparameter V1

struct disp_info 占用存储空间： 8192 Byte（有效数据为 7158 Byte）；

struct baseparameter_info 合计占用存储空间：2 x 8192 Byte = 16384 Byte；

Baseparameter 主分区和备份分区的大小均固定为 512 KB，目前用到的有效数据是 16384 Byte（16 KB），剩下未使用的空间预留。

3.4.2 Baseparameter V2

char head_flag[4] 占用存储空间： 4 Byte；

u16 major_version 占用存储空间： 2 Byte；

u16 minor_version 占用存储空间： 2 Byte；

struct disp_header 占用存储空间： 12 Byte；

strcut disp_info 占用存储空间： 36932 Byte；

struct pq_tuning_info 占用存储空间： 15557 Byte；

struct pq_factory_info 占用存储空间： 90 Byte；

struct pq_sharp_info 占用存储空间：43 Byte；

struct aipq_info 占用存储空间：63 Byte

struct baseparameter_info 合计占用存储空间：4 + 2 + 2 + 8 x 12 + 8 x 36932 + 15557 + 90 + 43 + 63 = 311313 Byte；

Baseparameter 主分区和备份分区的大小均固定为 512 KB，目前用到的有效数据是 311313 Byte（304.02 KB），剩下未使用的空间预留。

3.4.3 Baseparameter V3

char head_flag[4] 占用存储空间： 4 Byte；

u16 major_version 占用存储空间： 2 Byte；

u16 minor_version 占用存储空间： 2 Byte；

u8 disp_num 占用存储空间： 1 Byte；

u32 reserved_offset 占用存储空间： 4 Byte；

struct disp_header 占用存储空间： 12 Byte；

strcut disp_info 占用存储空间： 51721 Byte；

struct baseparameter_info 合计占用存储空间：4 Byte + 2 Byte + 2 Byte + 1 Byte + 4 Byte + (disp_num) x 12 Byte + (disp_num) x 51721 Byte；

Baseparameter 主分区和备份分区的大小相同，具体由 Baseparameter 分区中记录的显示通路数量决定。

4. Baseparameter 备份分区

Baseparameter 的后半部分用来保存初始数据，恢复出厂设置的时候将备份分区的数据拷贝到主分区。

5. Baseparameter 读写 API 说明

为方便应用层对 Baseparameter 分区的读写，将以库的形式提供统一的 API 供不同的应用调用。
库的源码位置在：hardware/rockchip/libbaseparameter
API 说明：

```
bool have_baseparameter(); /* 判断是否有baseparameter分区 */
int dump_baseparameter(const char *file_path); /* 导出baseparameter分区数据到文件
*/
int get_disp_info(unsigned int connector_type, unsigned int connector_id, struct
disp_info *info); /* 获取disp_info */
int set_disp_info(unsigned int connector_type, unsigned int connector_id, struct
disp_info *info); /* 设置disp_info */
int get_screen_info(unsigned int connector_type, unsigned int connector_id, int
index, struct screen_info *screen_info); /* 获取screen_info */
int set_screen_info(unsigned int connector_type, unsigned int connector_id, int
index, struct screen_info *screen_info); /* 设置screen_info */
unsigned short get_brightness(unsigned int connector_type, unsigned int
connector_id); /* 获取亮度 */
unsigned short get_contrast(unsigned int connector_type, unsigned int
connector_id); /* 获取对比度 */
unsigned short get_saturation(unsigned int connector_type, unsigned int
connector_id); /* 获取饱和度 */
unsigned short get_hue(unsigned int connector_type, unsigned int connector_id);
/* 获取色调 */
int set_brightness(unsigned int connector_type, unsigned int connector_id,
unsigned short value); /* 设置亮度 */
int set_contrast(unsigned int connector_type, unsigned int connector_id,
unsigned short value); /* 设置对比度 */
int set_saturation(unsigned int connector_type, unsigned int connector_id,
unsigned short value); /* 设置饱和度 */
int set_hue(unsigned int connector_type, unsigned int connector_id, unsigned
short value); /* 设置色调 */
int get_overscan_info(unsigned int connector_type, unsigned int connector_id,
struct overscan_info *overscan_info); /* 获取overscan_info */
int set_overscan_info(unsigned int connector_type, unsigned int connector_id,
struct overscan_info *overscan_info); /* 设置overscan_info */
int get_gamma_lut_data(unsigned int connector_type, unsigned int connector_id,
struct gamma_lut_data *data); /* 获取gamma数据 */
int set_gamma_lut_data(unsigned int connector_type, unsigned int connector_id,
struct gamma_lut_data *data); /* 设置gamma数据 */
int get_cubic_lut_data(unsigned int connector_type, unsigned int connector_id,
struct cubic_lut_data *data); /* 获取3dlut数据 */
int set_cubic_lut_data(unsigned int connector_type, unsigned int connector_id,
struct cubic_lut_data *data); /* 设置3dlut数据 */
int set_disp_header(unsigned int index, unsigned int connector_type, unsigned
int connector_id); /* 设置connector_id，connector_type和分区中disp_info的对应关系 */
bool validate(); /* 判断分区数据是否有效 */
int get_framebuffer_info(unsigned int connector_type, unsigned int connector_id,
framebuffer_info *info); /* 获取framebuffer数据 */
int set_framebuffer_info(unsigned int connector_type, unsigned int connector_id,
framebuffer_info *info); /* 设置framebuffer数据 */
int get_baseparameter_info(unsigned int index, baseparameter_info *info); /* 获取
baseparameter数据 */
int set_baseparameter_info(unsigned int index, baseparameter_info *info); /* 设置
baseparameter数据 */
int set_pq_tuning_info(struct pq_tuning_info *info); /* 获取pq tuning数据 */
int get_pq_tuning_info(struct pq_tuning_info *info); /* 设置pq tuning数据 */
int get_csc_info(struct csc_info *csc); /* 获取csc数据 */
```

```
int set_csc_info(struct csc_info *csc); /* 设置csc数据 */
int get_dci_info(struct dci_info *dci); /* 获取dci数据 */
int set_dci_info(struct dci_info *dci); /* 设置dci数据 */
int get_acm_info(struct acm_info *acm); /* 获取acm数据 */
int set_acm_info(struct acm_info *acm); /* 设置acm数据 */
int get_pq_tuning_gamma(struct gamma_lut_data *data); /* 获取pq tuning gamma数据
*/
int set_pq_tuning_gamma(struct gamma_lut_data *data); /* 设置pq tuning gamma数据
*/
int get_pq_factory_info(struct pq_factory_info *info); /* 获取pq工厂矫正数据 */
int set_pq_factory_info(struct pq_factory_info *info); /* 设置pq工厂矫正数据 */
int get_pq_sharp_info(struct pq_sharp_info *info); /* 获取sharp数据 */
int set_pq_sharp_info(struct pq_sharp_info *info); /* 设置sharp数据 */
int get_version(unsigned short* major_version, unsigned short* minor_version);
/* 获取baseparameter版本号 */
int get_aipq_info(struct aipq_info *info); /* 获取aipq数据 */
int set_aipq_info(struct aipq_info *info); /* 设置aipq数据 */
```

6. Baseparameter 解析工具使用说明

saveBaseParameter 是 Android上 用来对 baseparameter 分区管理工具，实现了打印、修改分区数据等功能。
工具的源码位置：device/rockchip/common/baseparameter/saveBaseParameter
使用说明：

```
saveParameter: read and write baseparameter partition tool

Usage:
        -h      Help info
        -p      Print Baseparamter
        -t      output to target file (e: "/sdcard/baseparameter.img)"
        -f      Framebuffer Resolution (e: 1920x1080@60)
        -c      Color (e: RGB-8bit or YCBCR444-10bit)
        -u      Is Enable Auto Resolution (auto resolution:"auto";set one fixed
resolution:hdisplay,vdisplay,vrefresh,hsync_start,hsync_end,htotal,vsync_start,v
sync_end,vtotal,vscan,flags,clock e:
"1920,1080,60,2008,2052,2200,1084,1089,1125,0,5,148500")
        -o      Overscan (left,top,right,bottom e: overscan "100,100,100,100")
        -b      BCSH (brightness,contrast,saturation,hue e: "50,50,50,50")
        -R      Reset Baseparameter (1:only reset user setting baseparameter
partition; 2:reset baseparameter paratition include backup)
        -C      Choose Connector type and id to Setting (e: 11,0 or 16,0)

Example: saveBaseParameter -C "16,0" -f "1920x1080@60" -c Auto -u 2 -o
"100,100,100,100" -b "50,50,50,50"
```

7. Android 软件配置流程和代码路径

DisplayAjust源码位置：packages/apps/DisplayAdjust

流程图：

